

```
1.  /**
2.   * @file Servicio para las operaciones de contactos de emergencia
3.   * @description Proporciona métodos para interactuar con la API de contactos de emergencia
4.   * @requires ../config/api.service
5.   */
6.  import api from '../config/api.service';
7.
8.  /**
9.   * @namespace contactoEmergenciaService
10.   * @description Un objeto que agrupa todos los métodos de servicio para los contactos de emergencia
11.   */
12.  export const contactoEmergenciaService = {
13.    /**
14.     * @function getAll
15.     * @description Obtiene todos los contactos de emergencia del sistema
16.     * @returns {Promise<Array<object>>} Una lista de los contactos de emergencia
17.     * @async
18.     */
19.    getAll: async () => {
20.      const { data } = await api.get('/api/contactos-emergencia');
21.      return data;
22.    },
23.
24.    /**
25.     * @function create
26.     * @description Crea un nuevo contacto de emergencia.
27.     * @param {object} contactData - Datos del contacto (nombre, teléfono, dirección)
28.     * @returns {Promise<object>} El contacto creado.
29.     * @async
30.     */
31.    create: async (contactData) => {
32.      const { data } = await api.post('/api/contactos-emergencia', contactData);
33.      return data;
34.    },
35.
36.    /**
37.     * @function sendEmergencyEmail
38.     * @description Envía un email de emergencia a un contacto.
39.     * @param {string} contactoId - ID del contacto de emergencia
40.     * @returns {Promise<object>} Respuesta del servidor.
41.     * @async
42.     */
43.    sendEmergencyEmail: async (contactoId) => {
```

Home

Namespaces

apiConfig

contactoEmergenciaService

diarioService

Classes

ErrorBoundary

Events

SIGINT

unhandledRejection

Global

404_Error_Handler

ACTIVIDADES

API_URL

ASPECTOS_COGNITIVOS

ActionCard

ActivitySelector

App

AuthButtons

AuthLayout

Button

CORS_Configuration

Card

Carousel

Checkbox

CircularProgress

CognitionSelector

DiaryBanner

DiaryEditor

DiaryEntry

EMOCIONES

EmergencyButton

EmergencyModal

EmotionSelector

EmotionStats

Footer

GET_/_

Generic_Error_Handler

Heading

HeroSection

Home

Input

Landing

LandingHero

Login

Logo

MainLayout

Navbar

ProtectedRoute

Register

Seguimiento

Slider

Text

```
44.         const { data } = await api.post(`/api/contactos-emergenci
45.         return data;
46.     },
47.     );
48.
```

Textarea
accessWithPassword
actualizarEntradaDiario
api
apiFetch
authMiddleware
closeShareModal
compararPassword
componentDidCatch
connectDBAndStartServer
copyShareLink
crearEntradaDiario
create
createContacto
createRegistro
delete
deleteContacto
description
dismiss
duracion
eliminarEntradaDiario
emotion
error
formatDate
formatDateShort
getAll
getById
getContactoById
getContactos
getDerivedStateFromError
getIntensidad
getPreviewText
getProfile
getRegistroByFecha
getRegistroById
getRegistros
getRegistrosByRango
getStepLabel
goToNext
goToPrevious
handleAddActividad
handleAddContact
handleCall
handleCancelAdd
handleCancelEditor
handleChange
handleCreateEntry
handleDeleteEntry
handleEditEntry
handleEmocionToggle
handleInputChange
handleIntensidadChange
handleLogin
handleLogout
handleOverlayClick
handleRegister
handleReload
handleRemoveActividad
handleSendEmail
handleSendSms
handleShareEntry

handleSubmit
handleToggle
handleToggleExpand
handleUpdateEntry
info
intensidad
isMobileDevice
isSelected
loadContacts
loadEntries
loading
loginUser
nextStep
nombre
obtenerEntradaDiarioPorId
obtenerEntradasDiario
optionalAuthMiddleware
percentage
port
pre-save
prevStep
promise
registerUser
render
request-interceptor
response-interceptor
sendEmergencyEmail
success
timeAgo
title
togglePasswordField
update
updateContacto
uri
useAuthStore
useToast
validateRequiredEnvVars